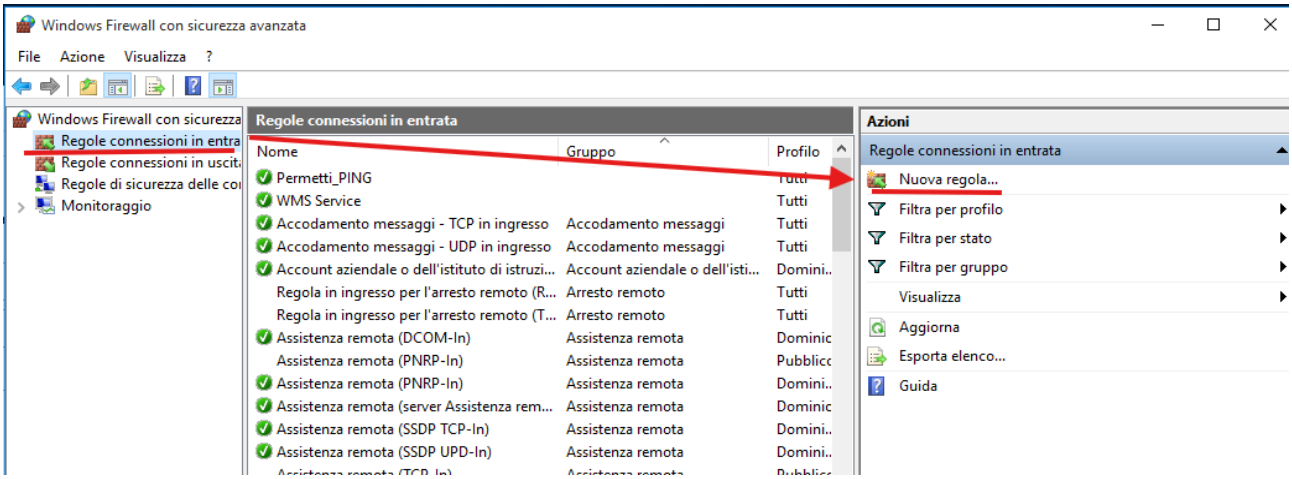
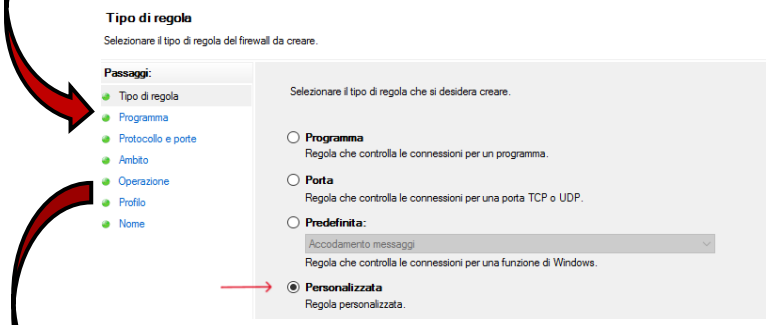
Alessandro Accordino

Configurazione Policy Firewall Windows e Wireshark

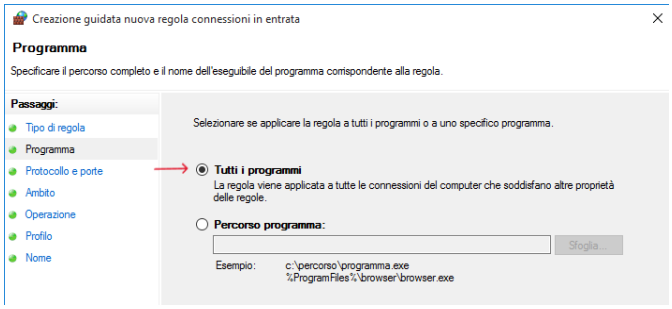
1)Configurazione Policy: In questa esercitazione andremo a configurare una Policy ovvero una nuova regola per permettere la comunicazione tra la VM windows e le altre VM che, come abbiamo visto, non riescono a comunicare per via del firewall di windows che blocca ogni tentativo di accesso.

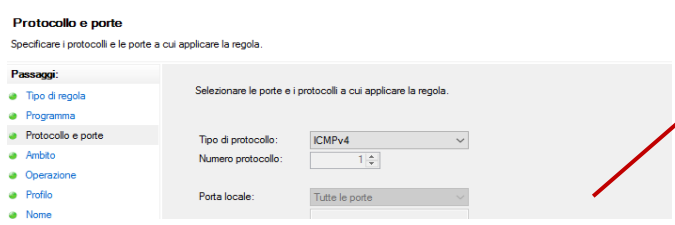
Per risolvere questo problema dobbiamo configurare una regola che permetta l'accesso a quegli IP, per fare ciò andiamo sulla nostra macchina virtuale windows: **barra di ricerca>"windows firewall">impostazioni avanzate>Regole di connessioni in entrata e clicchiamo su "Nuova Regola"**

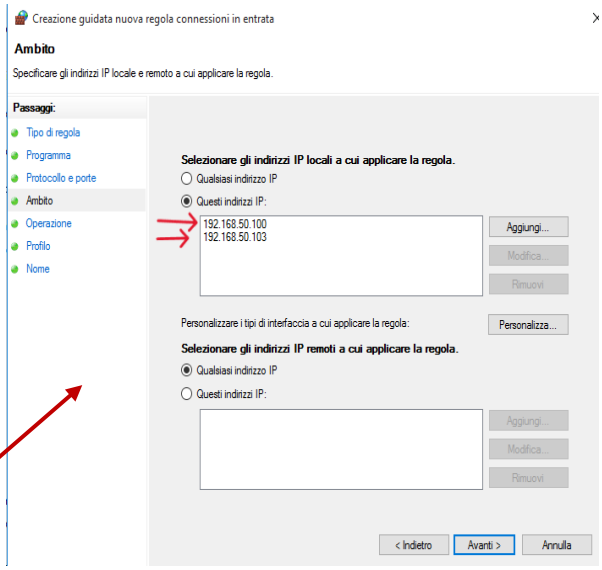




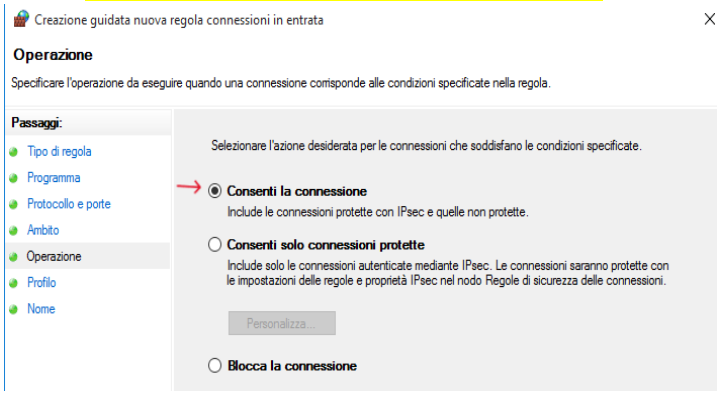
Successivamente Clicchiamo su "Personalizzata">"Tutti i programmi">"ICMP v4">"Questi indirizzi IP" e inseriamo il 192.168.50.100 di Kali linux e il 192.168.50.103 di Ubuntu:

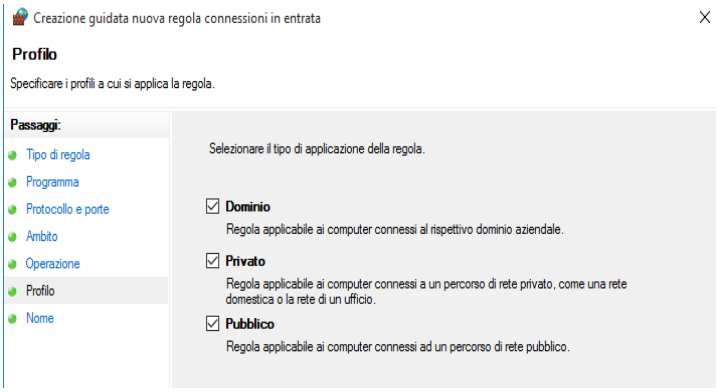




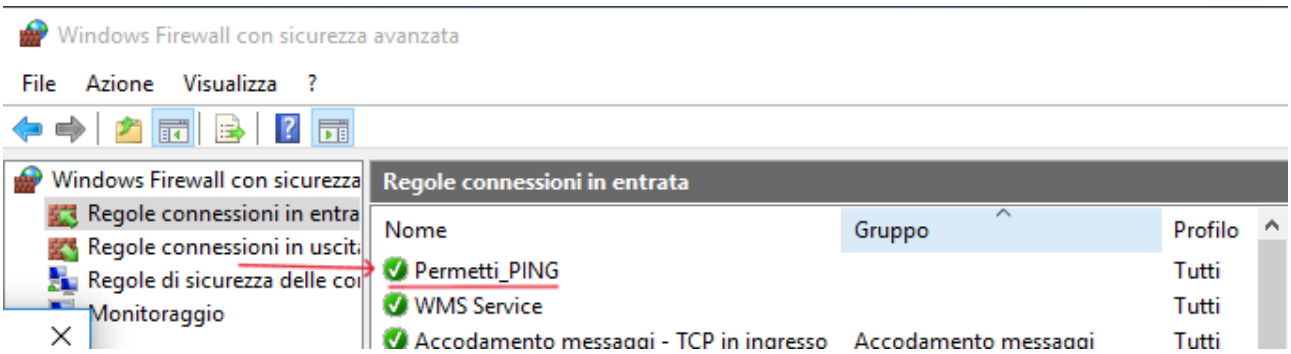


Successivamente facciamo sempre avanti e “Consenti la connessione” e spuntiamo “Dominio”,”Privato” e “Pubblico”

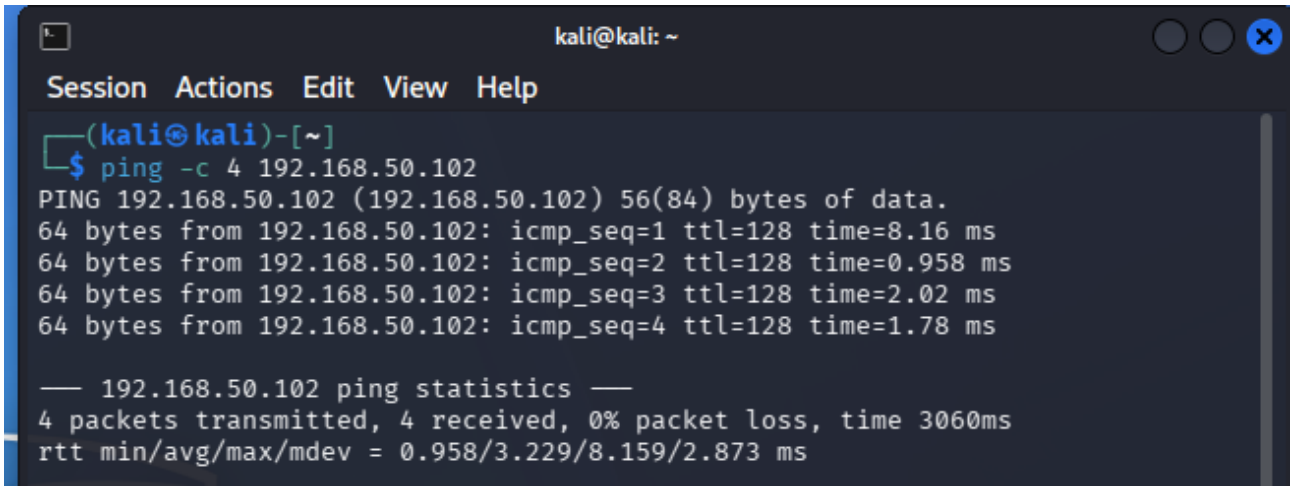




Per ultima Inseriamo il Nome della regola e se vogliamo una descrizione e facciamo “Fine”. In questo modo la nostra regola sarà aggiunta in cima:



Prova di connetivittà: Adesso proviamo ad effettuare un Ping con la VM Linux per verificare che la regola sia funzionante e il firewall non scarti nessun pacchetto:



Come possiamo vedere la comunicazione avviene senza problemi e dei 4 pacchetti inviati tutti e 4 vengono consegnati con un TTL di 128 dato che fanno parte della stessa rete.

Wireshark

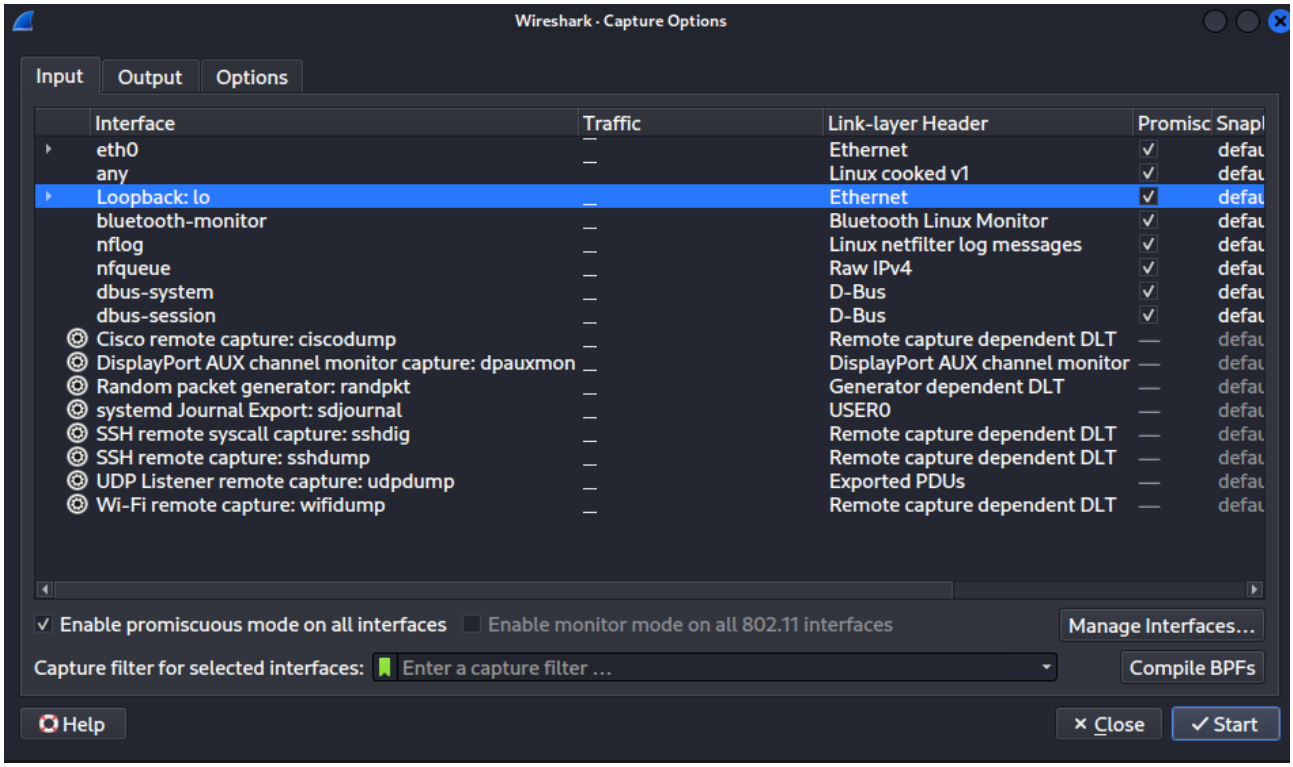
2)Configurazione di InetSim. Per emulare vari servizi di rete e per provare il funzionamento di wireshark nello sniffare i pacchetti andiamo a configurare dal terminale di linux InetSim che ci permette di fare ciò. Per prima cosa andiamo a commentare tutti i servizi e lasceremo attivo solo il servizio HTTPS scrivendo **"sudo nano /etc/inetsim/inetsim.conf"**:

```
kali@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.7.1              /etc/inetsim/inetsim.conf *
# Syntax: start_service <service name>
#
# Default: none
#
# Available service names are:
# dns, http, smtp, pop3, tftp, ftp, ntp, time_tcp,
# time_udp, daytime_tcp, daytime_udp, echo_tcp,
# echo_udp, discard_tcp, discard_udp, quotd_tcp,
# quotd_udp, chargen_tcp, chargen_udp, finger,
# ident, syslog, dummy_tcp, dummy_udp, smtps, pop3s,
# ftps, irc, https
#
#start_service dns
#start_service http
start_service https
#start_service smtp
#start_service smtps
#start_service pop3
#start_service pop3s
#start_service ftp
#start_service ftps
#start_service tftp
#start_service irc
#start_service ntp
Save modified buffer?
 Y Yes
 N No           ^C Cancel
```

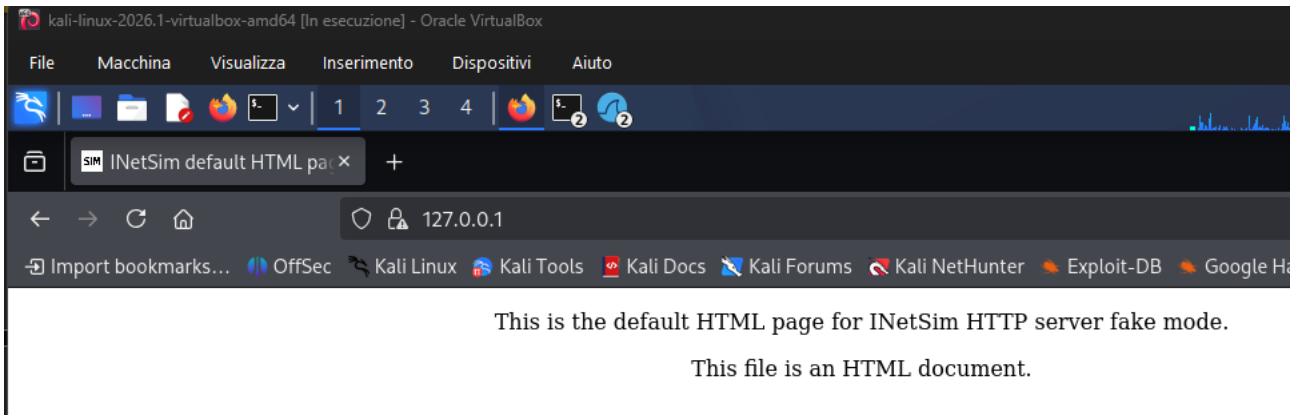
Fatto ciò salviamo facendo ctrl+x> Y > invio. Adesso per avviare il servizio ci basterà scrivere nel terminale "sudo inetsim"

```
┌──(kali㉿kali)-[~]
└─$ sudo inetsim
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 9416) ===
Session ID:     9416
Listening on:   127.0.0.1
Real Date/Time: 2026-04-24 09:27:09
Fake Date/Time: 2026-04-24 09:27:09 (Delta: 0 seconds)
 Forking services...
  * https_443_tcp - started (PID 9418)
^X@sS done.
Simulation running.
```

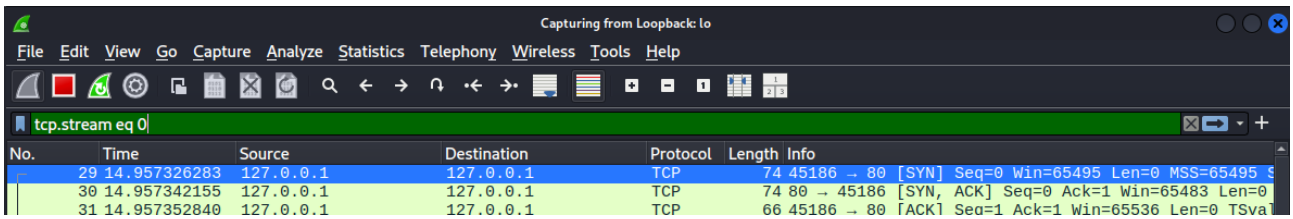
Fatto ciò possiamo avviare wireshark scrivendo nel terminale “sudo wireshark”. Successivamente selezioniamo come intefaccia “Loopback:lo”> start:



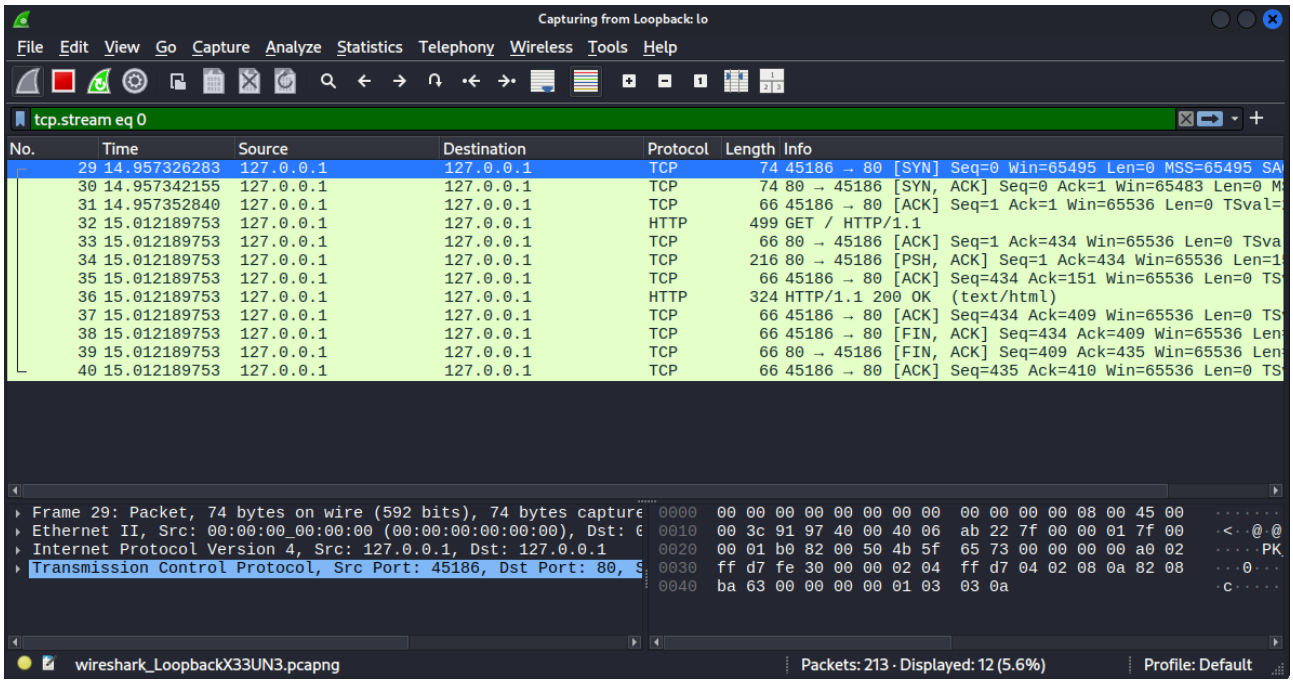
Dopo aver avviato andiamo sul web e scriviamo 127.0.0.1 ovvero la fake page creata da Inetsim per emulare il servizio https:



Fatto ciò ritorniamo su wireshark e applichiamo il filtro “tcp.stream eq 0” che si occupa di catturare solamente il primo flusso di pacchetti TCP:



Come possiamo vedere se Analizziamo la comunicazione catturata da wireshark possiamo vedere un three way handshake per la corretta sicronizzazione e comunicazione(syn, syn+ack, ack).



I 3 passaggi

1. **SYN**: il client dice “voglio aprire la connessione” e propone i parametri.
2. **SYN+ACK**: il server risponde “ricevuto, accetto e sincronizzo”.
3. **ACK**: il client conferma e la connessione è pronta per scambiarsi i dati.